

2026-05-05-p1-f07-background-task-system-design

Phase 1 / Feature 7 — Background Task System (Ctrl+B)

Date: 2026-05-05 **Status:** Approved (brainstorming) **Programme:** CLI-Agent Fusion — Phase 1
port from claude-code

1. Goal

Add background-task execution to HelixCode so the agent (and user) can fire long-running tools — shell builds, test suites, package installs — into goroutines without blocking the conversation. Tools invoked with `run_in_background: true` return a task ID immediately and continue executing asynchronously. New `TaskOutput` and `TaskStop` agent tools let the agent inspect tail output and cancel tasks. A `/tasks` slash command exposes the same view to users in interactive sessions.

This ports claude-code's Ctrl+B background pattern into HelixCode, scoped to in-memory ephemeral execution with line-oriented progress streaming for shell-style tools.

2. Architecture

Extend `internal/workflow/` with a new `background.go` file holding `BackgroundManager` and `BackgroundTask` (sibling to the existing multi-step `Executor/Workflow` machinery). Tools that produce line-oriented output opt into a new `BackgroundAware` interface; everything else falls back to “final result only” semantics — no `Tool` interface changes for tools that don't opt in. The `ToolRegistry.Execute` dispatcher (already the central hook point post-F05) detects `run_in_background: true` in params, strips the flag, and routes to `BackgroundManager.StartTask`. A sweeper goroutine inside `BackgroundManager` reaps completed tasks older than 1h every 5 minutes.

Boundary discipline: - `BackgroundManager` knows about generic `func(ctx, args, sink) (any, error)` executors — no dependency on `tools.Tool`. The registry adapts. The workflow package stays orthogonal to the tools package. - `BackgroundAware` is a small additive interface in `internal/tools/`; tools that don't implement it work unchanged. - `/tasks` slash command and the `TaskOutput/TaskStop` agent tools are sibling consumers of the same `BackgroundManager`. Neither calls the other.

3. Components

3.1 New files

File	Responsibility
helix_code/internal/workflow/background.go	BackgroundManager, BackgroundTask, TaskState, ManagerConfig, sweeper
helix_code/internal/workflow/background_test.go	Unit tests for the manager + task
helix_code/internal/tools/task_tools.go	TaskOutputTool, TaskStopTool (agent-callable)
helix_code/internal/tools/task_tools_test.go	Unit tests for the agent tools
helix_code/internal/tools/shell/background.go	BackgroundAware adapter for the shell tool
helix_code/internal/tools/shell/background_test.go	Unit tests for the shell adapter
helix_code/internal/commands/tasks_command.go	/tasks slash command
helix_code/internal/commands/tasks_command_test.go	Unit tests for the slash command
helix_code/tests/integration/background_shell_test.go	Real-subprocess integration tests (/go:build integration)
challenges/p1-f07-background-tasks/CHALLENGE.md	Runtime-evidence Challenge spec
challenges/p1-f07-background-tasks/run.sh	Challenge runner

3.2 Modified files

File	Change
helix_code/internal/tools/registry.go	Execute detects run_in_background: true, calls BackgroundManager.StartTask; new SetBackgroundManager(*workflow.BackgroundManager) and RegisterTaskTools(*workflow.BackgroundManager) methods
helix_code/internal/	New RegisterBuiltinCommandsWithTasks(registry, mgr) mirroring the Hooks/MCP pattern

File	Change
commands/ builtin/ register.go	
helix_code/ cmd/cli/ main.go	Construct BackgroundManager, wire into ToolRegistry + RegisterBuiltinCommandsWithTasks

3.3 BackgroundAware interface (internal/tools/)

```

type LineSink func(line string)

type BackgroundAware interface {
    Tool
    ExecuteWithProgress(ctx context.Context, params
        map[string]any, sink LineSink) (any, error)
}

```

Implementations call `sink(line)` for each line of progress output. The non-streaming `Execute` path remains as the fallback for foreground use. Tools that don't implement `BackgroundAware` get final-result-only behavior under `run_in_background: true`.

3.4 BackgroundTask and BackgroundManager

```

type TaskState string
const (
    TaskPending    TaskState = "pending"
    TaskRunning    TaskState = "running"
    TaskCompleted TaskState = "completed"
    TaskFailed     TaskState = "failed"
    TaskCancelled  TaskState = "cancelled"
)

type BackgroundTask struct {
    ID          string
    ToolName    string
    Args        map[string]any
    StartedAt   time.Time
    EndedAt     *time.Time
    // private: state (atomic.Int32), output ring, result, err,
    //           ctx, cancel, mu
}

func (bt *BackgroundTask) State() TaskState
func (bt *BackgroundTask) LastLines(n int) []string
func (bt *BackgroundTask) AppendOutput(line string)
func (bt *BackgroundTask) Result() (any, error)

```

```

type ManagerConfig struct {
    OutputCap      int           // per-task ring; default 256
    SweepInterval time.Duration // default 5*time.Minute
    MaxAge         time.Duration // default 1*time.Hour
    MaxConcurrent  int           // default 64
    LineBytesMax   int           // per line; default 4096
}

type BackgroundManager struct {
    // tasks map, mu, sweeper closeCh, closed, cfg, log
}

func NewBackgroundManager(log *zap.Logger, cfg ManagerConfig)
    *BackgroundManager
func (bm *BackgroundManager) StartTask(toolName string, args
    map[string]any,
    exec func(ctx context.Context, args map[string]any, sink
        LineSink) (any, error)) (*BackgroundTask, error)
func (bm *BackgroundManager) GetTask(id string)
    (*BackgroundTask, error)
func (bm *BackgroundManager) StopTask(id string) error
func (bm *BackgroundManager) ListTasks() []*BackgroundTask
func (bm *BackgroundManager) Status(id string) (TaskState,
    []string, error)
func (bm *BackgroundManager) Close() error

```

The exec closure passed to `StartTask` receives a sink. The registry's adapter constructs that closure from a `tools.Tool`: if the tool implements `BackgroundAware`, the closure calls `ExecuteWithProgress`; otherwise it calls `Execute` and writes the formatted result as a single sink line at the end.

3.5 Registry dispatch path

```

// In ToolRegistry.Execute, near the top:
if bg, ok := params["run_in_background"].(bool); ok && bg {
    if r.bgManager == nil {
        return nil, ErrNoBackgroundMgr
    }
    cleanArgs := stripBackgroundFlag(params)
    tool, err := r.Get(name)
    if err != nil { return nil, err }
    bgExec := r.adaptToolForBackground(tool)
    task, err := r.bgManager.StartTask(name, cleanArgs, bgExec)
    if err != nil { return nil, err }
    return map[string]any{
        "task_id": task.ID,
        "state":   string(task.State()),
        "message": fmt.Sprintf("Task started in background. ID:
            %s – use TaskOutput to check progress.", task.ID),
    },
}

```

```
    }, nil
}
```

3.6 Agent tools

- TaskOutputTool — reads `task_id` (required) + `lines` (optional, default 5); returns `{task_id, state, output, line_count, total_lines}`.
- TaskStopTool — reads `task_id`; cancels the task; returns `{task_id, status: "stopped"}`.

Both implement the standard `tools.Tool` interface.

3.7 Shell BackgroundAware adapter

Wraps the existing shell tool. Uses `exec.Cmd.StdoutPipe/StderrPipe`, scans line-by-line via `bufio.Scanner`, calls `sink(line)` for each. Final result mirrors the foreground shell tool. Sandbox/security checks reused unchanged.

3.8 /tasks slash command

Subcommands: `- /tasks` or `/tasks list` — table `NAME / ID / STATE / TOOL / STARTED`. `- /tasks output <id>` — last 20 lines. `- /tasks stop <id>` — cancels the task.

Mirrors `/permissions`, `/worktree`, `/hooks`, `/mcp` patterns.

4. Data flow

4.1 Foreground call (default — backward-compatible)

```
agent calls "Bash" with params={"command": "ls"}
└─ ToolRegistry.Execute(ctx, "Bash", params)
    └─ no run_in_background flag → existing path
        └─ fireBefore (hooks)
            └─ tool.Execute(ctx, params)
                └─ fireAfter
                    └─ return result
```

4.2 Background call

```
agent calls "Bash" with params={"command": "go test ./...", "run_in_backgr
└─ ToolRegistry.Execute(ctx, "Bash", params)
    └─ detects run_in_background=true; strips flag
        └─ tool := r.Get("Bash")
            └─ exec := r.adaptToolForBackground(tool)
                └─ implements BackgroundAware → wraps tool.ExecuteWithProgres
                    └─ otherwise → wraps tool.Execute, sink called once at comple
            └─ task := bgMgr.StartTask("Bash", cleanArgs, exec)
                └─ state=Pending
                    └─ goroutine: state=Running → exec(ctx, args, AppendOutput) →
    └─ return {task_id, state: "pending", message} immediately
```

4.3 Polling (TaskOutput)

```
TaskOutputTool.Execute({task_id, lines=10})
├─ task := bgMgr.GetTask(id)
├─ output := task.LastLines(10)
└─ return {task_id, state, output (newline-joined), line_count, total_li
```

4.4 Cancellation (TaskStop or /tasks stop)

```
bgMgr.StopTask(id)
├─ if state ∉ {Running, Pending} → ErrTaskNotRunning
├─ task.cancel() // ctx propagates to executor
├─ shell exec sees ctx.Done → SIGKILL (via exec.CommandContext)
└─ executor goroutine sets state=Cancelled, EndedAt=now
```

StopTask returns once cancel is called; the goroutine writes the terminal state independently to avoid races on task.state.

4.5 Sweeper

```
ticker every cfg.SweepInterval
├─ for each task:
│   if state ∈ {Completed, Failed, Cancelled} and now > *EndedAt + cfg.
│       delete tasks[id]
```

In-flight tasks are never reaped.

4.6 Shutdown (Close)

```
mu.Lock; if closed → return nil; closed=true; close(closeCh); snapshot in-
for each in-flight: task.cancel()
wait briefly (5s) for goroutines to drain
```

4.7 Hooks

Foreground: fireBefore/fireAfter already wrap tool.Execute. Background: hooks fire only on the dispatch event (the synchronous Execute call that returns task_id). Completion runs in a goroutine outside the hooks' scope. A future feature could add OnBackgroundTaskComplete hooks; deferred.

5. Error handling

5.1 Error sentinels

```
var (
    ErrTaskNotFound      = errors.New("workflow: background task
        not found")
    ErrTaskNotRunning    = errors.New("workflow: task is not
        running")
    ErrManagerClosed     =
        errors.New("workflow: background manager closed")
```

```

ErrNoBackgroundMgr = errors.New("tools: run_in_background
    requested but no BackgroundManager wired")
ErrTooManyTasks    = errors.New("workflow: too many
    concurrent background tasks")
)

```

5.2 Failure modes

- **Panic in goroutine:** recovered via `defer recover()`; converted to error; state=Failed; `AppendOutput("panic: ...")`.
- **Tool returns (nil, nil):** state=Completed; `Result()=(nil, nil)`.
- **ctx cancelled mid-execution:** executor exits with `ctx.Err()`; state=Cancelled.
- **Tool not in registry:** errors before `StartTask`; no phantom task created.
- **Sweeper deletes a task with a held reference:** pointer remains valid (Go GC); subsequent `GetTask(id)` returns `ErrTaskNotFound`.

5.3 Concurrency invariants

- One executor goroutine per task; state writes serialised through `BackgroundTask.SetState` under `bt.mu`. `State()` reads via `atomic.Int32`.
- `BackgroundManager.mu` (`sync.RWMutex`) guards `tasks map`. `StartTask` write-locks; `GetTask/ListTasks/Status` read-lock; `sweeper` write-locks.
- `task.AppendOutput` takes `bt.mu` for slice mutation. Output ring bounded at `cfg.OutputCap`; appends past the cap drop the oldest line; each line trimmed to `cfg.LineBytesMax`.
- Sweeper, executor, and `Close` coordinated via `closeCh` + per-task `ctx`. No deadlock paths (manager → task lock direction only).

5.4 Resource limits (defaults)

Limit	Default
OutputCap (lines per task)	256
LineBytesMax	4096
MaxConcurrent	64
SweepInterval	5 min
MaxAge	1 hour

Per-task memory upper bound: $256 \times 4 \text{ KB} = 1 \text{ MB}$. Manager upper bound: $64 \times 1 \text{ MB} = 64 \text{ MB}$ worst case.

5.5 Anti-bluff (CONST-035, Article XI §11.9)

- Challenge in T13 spawns real shell command, polls `TaskOutput` across multiple intervals, captures the actual streaming timeline.
- `TaskOutput` returns real content from `task.output`, never a placeholder string. State carries the running/done signal; output carries lines.
- `TaskStop` actually cancels the subprocess. Challenge verifies no orphan PID via `pgrep`.
- No mock subprocess in integration tests — real shell, real exec, real PID lifecycle.

- Anti-bluff smoke `grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" internal/workflow/ internal/tools/task_tools.go internal/tools/shell/background.go internal/commands/tasks_command.go` must return empty.

5.6 Logging

- `StartTask` logs INFO with id + tool name.
- State transitions log INFO.
- Errors log WARN with id + error.
- Sweeper deletions log DEBUG.
- No tool params or output content logged (could contain secrets).

6. Testing

6.1 Unit tests (mocks allowed)

`internal/workflow/background_test.go`: - `StartTaskRunsExecutor`, `StopTaskCancelsCtx`, `StopUnknownTaskReturnsError`, `StopAlreadyDoneTaskRejects`, `GetTaskMissing`, `OutputRingBoundedAtCap`, `OutputLineTruncatedAt4KB`, `PanickingExecRecovers`, `NilResultNilError`, `MaxConcurrentEnforced`, `SweeperReapsCompleted`, `SweeperLeavesRunning`, `CloseIdempotent`, `CloseCancelsInFlight`.

`internal/tools/task_tools_test.go`: - `TaskOutputTool_ReturnsLastNLines`, `TaskOutputTool_DefaultLinesIs5`, `TaskOutputTool_UnknownTaskID`, `TaskStopTool_CancelsRunning`, `TaskStopTool_NotRunningRejects`.

`internal/tools/shell/background_test.go`: - `ShellBackgroundAware_StreamsLines`, `ShellBackgroundAware_ContextCancelKillsProcess`, `ShellBackgroundAware_ExitNonZeroIsErrorButCompletes`.

`internal/tools/registry_background_test.go`: - `RunInBackgroundFlagDispatchesToManager`, `RunInBackgroundWithoutManagerErrors`, `BackgroundFlagStrippedFromArgs`, `NonBackgroundAwareUsesFallback`.

`internal/commands/tasks_command_test.go`: - `SlashTasks_ListEmpty`, `SlashTasks_ListShowsTask`, `SlashTasks_OutputReturnsLines`, `SlashTasks_StopCancels`, `SlashTasks_UnknownSubcommand`.

6.2 Integration tests (-tags=integration, real subprocess)

`tests/integration/background_shell_test.go`: - `TestBackground_ShellEcho_StreamsAndCompletes` — full path through real registry, real `bgMgr`, real subprocess; assert “hi” in output, state=Completed. - `TestBackground_ShellSleep_StopCancels` — real `sleep 30`, `TaskStop`, assert state=Cancelled within 2s, no orphan PID via `pgrep`. - `TestBackground_ConcurrentTasks` — 5 parallel `sleep 1` tasks; all reach Completed; sweeper reduces to 0.

All -tags=integration; no bare `t.Skip()` without SKIP-OK: <ticket>.

6.3 Challenge

challenges/p1-f07-background-tasks/:

1. Build `bin/cli`.
2. Small Go harness fires Bash with command: `"for i in 1 2 3; do echo line $i; sleep 0.5; done"` and `run_in_background: true`.
3. Poll `TaskOutput` at 250ms intervals; capture sequence: empty → 1 line → 2 → 3 → state=Completed. Mid-execution streaming captured (not just final result).
4. Second Bash background `sleep 30`; wait 200ms; `TaskStop`; verify cancelled within 1s; verify no orphan `sleep` process.
5. Anti-bluff smoke clean.
6. Cross-compile linux.

Pasted runtime evidence committed to `docs/improvements/06_phase_1_evidence.md` per Article XI §11.9.

7. Cross-platform

Shell `BackgroundAware` uses `os/exec + pipes + bufio.Scanner + exec.CommandContext` for ctx-driven cancellation. The existing shell sandbox/security infrastructure already handles process-group concerns. No new platform-specific code needed for F07. Cross-compile check (Linux native; Windows pre-existing CGO failures in unrelated packages remain documented out-of-scope).

8. Out of scope (deferred)

- `BackgroundAware` adapters for browser, web, MCP tools. Documented in F07-T01 evidence as “follow-up if usage warrants”.
- Persistence of completed tasks across CLI sessions. The `internal/task/` heavy machinery covers durable distributed work; F07’s `BackgroundManager` is intentionally ephemeral.
- Push notifications when output arrives. The agent polls via `TaskOutput`.
- `OnBackgroundTaskComplete` hooks. The hooks system fires on synchronous `Execute`; completion in a goroutine is outside that scope. Could be a follow-up.

9. Constitutional compliance

- **CONST-033** (no power management): no shell wrappers introduce shutdown/reboot vectors; cancellation only kills the subprocess group it started.
- **CONST-035 / Article XI §11.9** (anti-bluff): every PASS carries runtime evidence; the Challenge captures actual mid-execution polling output, not metadata.
- **CONST-036** (LLMsVerifier single source of truth): orthogonal — F07 doesn’t introduce model metadata.
- **CONST-039** (all-providers integration): orthogonal.
- **CONST-042** (no-secret-leak): tool params and output content are not logged. Secret-bearing tools (e.g., `mcp auth`) are not `BackgroundAware` in F07.
- **CONST-043** (no-force-push): branch is `main`; non-force pushes to all four remotes per programme convention.

10. Open questions resolved during brainstorming

Question	Answer
Q1: where does BackgroundManager live	(A) New file in internal/workflow/
Q2: output capture strategy	(B) Opt-in BackgroundAware interface; non-streaming tools fall back to final-result-only
Q3: user-facing surface	(B) /tasks slash command only (no top-level cobra)
Q4: which tools opt into BackgroundAware initially	(A) Shell only
Q5: cleanup policy	(A) Sweeper goroutine, defaults: 5min interval, 1h max age